# OASIS: Outlier-Aware LUT-Based GEMM with Dual-Side Quantization for LLM Inference Acceleration

Xueying Wu, Baijun Zhou, Zhihui Gao, Yuzhe Fu, Qilin Zheng, Yintao He†, Hai Li
*Duke University, Durham, NC, USA*
{xueying.wu, baijun.zhou, zhihui.gao, yuzhe.fu, qilin.zheng, yintao.he, hai.li}@duke.edu
†Corresponding author

***Abstract*—Large language models (LLMs) have demonstrated impressive capabilities across a wide range of applications, but demand substantial memory and compute resources during inference. Existing quantization methods expose a trade-off between efficiency and accuracy: weight-only quantization (WOQ) incurs costly dequantization overheads, while integer weight-and-activation quantization (INT-WAQ) reduces precision and degrades model quality. Non-uniform weight-and-activation quantization (NU-WAQ) can better capture the non-uniform distributions of LLM weights and activations, yet remains incompatible with conventional low-precision compute units.**

**This paper presents OASIS, a lookup table (LUT)-based architecture that enables efficient general matrix multiplication (GEMM) between non-uniformly quantized weights and activations without requiring dequantization. OASIS employs precomputed Cartesian Product LUTs, achieving a 64× reduction in LUT size and enabling a 1024× higher computational parallelism over existing LUT-based GEMM methods. To preserve accuracy under aggressive activation quantization, OASIS introduces an outlier-aware quantization scheme with concurrent LUT-based GEMM and error compensation for outliers. Furthermore, we design *Orizuru*, an efficient top-k detection engine for real-time activation outlier identification.**

**According to extensive evaluations, OASIS incurs an average accuracy drop of only 1.94% compared to the FP16 baseline, which is 6.34% lower than Atom. On the hardware side, OASIS achieves an average 3.00× speedup and a 1.44× energy efficiency improvement compared to the FIGLUT accelerator.**

***Index Terms*—large language models, non-uniform quantization, LUT-based computation, GEMM acceleration, hardware architecture, efficient inference**

## I. Introduction

Large language models (LLMs) have achieved strong performance across diverse domains such as dialogue systems [1], [17], code generation [46], [51], and electronic design automation [12], [18]. However, the rapidly increasing model sizes introduce substantial memory and computational costs during inference [5], [24], motivating extensive research on model compression.

Among these efforts, weight-only quantization (WOQ) [11], [29] stands out for effectively reducing memory footprint while preserving model accuracy. Yet WOQ still suffers from a key limitation: weights and activations reside in different numerical formats. Consequently, WOQ necessitates dequantizing weights to FP16 before executing, which is shown in Fig. 1(a). This dequantization step can dominate GEMM execution time, often constituting 20-90% of the total runtime [25], [30]. To eliminate the dequantization cost, recent studies [37], [42], [43] have proposed LUT-based GEMM methods that utilize lookup tables (LUTs) to directly execute GEMMs between quantized weight and FP16 activations. However, their performance remains constrained by the overhead of on-the-fly LUT generation, large LUT sizes and limited parallelism.

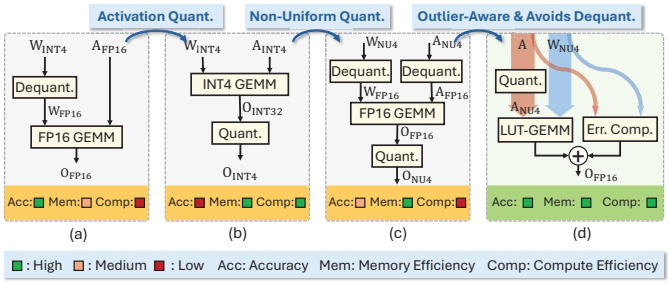


Fig. 1. Comparison of GEMM schemes: (a) $W_{INT4}A_{FP16}$ FP16 GEMM, (b) $W_{INT4}A_{INT4}$ INT4 GEMM, (c) $W_{NU4}A_{NU4}$ FP16 GEMM, (d) $W_{NU4}A_{NU4}$ LUT-GEMM (Ours). "NU" refers to non-uniform quantization. Our proposed design features two parallel computation branches that separately handle inlier and outlier operations. Our LUT-based GEMM design enables matrix multiplications between non-uniformly quantized weights and activations without requiring dequantization.

Weight-and-activation quantization (WAQ) offers a more promising approach by enabling fully low-precision GEMMs, reducing memory footprint for both weights and KV-cache memory, and fundamentally removing the need for mixed-format computation [3], [14], [33]. However, existing WAQ methods exhibit a fundamental trade-off between accuracy and compute efficiency. As shown in Fig. 1(b), integer WAQ (INT-WAQ) quantizes both operands into low-bit integers that existing low-precision hardware can directly process, but its limited representation capability often causes substantial accuracy degradation [58], [62]. In contrast, non-uniform WAQ (NU-WAQ), especially learned-codebook methods [19], [21], represents each value using an index that selects from a learned set of codebook entries, allowing the quantized values to more accurately follow the underlying data distribution. NU-WAQ significantly improves accuracy, but its index-coded data format is incompatible with existing low-precision compute units [40], [41]. As a result, NU-WAQ execution on existing hardware must dequantize values back to FP16 before execut-

979-8-3315-5065-3/26/$31.00 ©2026 IEEE
DOI 10.1109/ISCA66397.2026.00036

ing GEMMs, which is illustrated in Fig. 1(c). This negates the computational advantages of quantization and yields poor computational efficiency.

To resolve the dilemma between *high-efficiency but low-accuracy INT-WAQ* and *high-accuracy but low-efficiency NU-WAQ* (Fig. 1), we propose a LUT-based method that enables efficient GEMMs between non-uniformly quantized weights and activations without requiring dequantization. Prior LUT-based GEMM methods were developed to optimize GEMM computations with WOQ [37], [42], [43]. They rely on large inner-product LUTs that must be regenerated for dynamic activations, leading to excessive LUT size and limited parallelism. On the other hand, we observe that WAQ fundamentally unlocks three key opportunities to achieve a far more efficient LUT-based GEMM design: (1) In learned-codebook WAQ, both weight and activation centroids are learned offline, allowing the entire LUT to be pre-computed before inference and eliminating on-the-fly LUT generation. (2) Because both operands are quantized, the space of possible multiplication outcomes is greatly reduced, enabling us to store Cartesian Product entries instead of full inner products, which substantially shrinks LUT size. (3) The Cartesian Product LUT is independent of reduction length, enabling larger compute granularity and significantly higher parallelism during GEMMs.

To further retain model accuracy in WAQ, it is important to carefully handle the activation outliers. This is because activation outliers exhibit both higher quantity and magnitude compared to weight outliers, resulting in high quantization noises [3], [29], [58]. Therefore, we propose an outlier-aware mechanism that identifies activation outliers during inference and compensates for their errors without incurring additional runtime overhead.

Incorporating the aforementioned optimizations, we propose OASIS: an Outlier-Aware LUT-Based GEMM Scheme with Dual-Side Quantization for LLM Inference Acceleration. We compare the end-to-end LLM inference performance of OASIS with state-of-the-art (SOTA) quantization algorithms and accelerators. For the algorithmic performance, on average, OASIS achieves an accuracy degradation of only 1.94% compared to the FP16 baseline. This accuracy degradation is 6.34% lower than Atom [62]. For the hardware performance, OASIS achieves average speedups of 3.00× and energy efficiency improvements of 1.44× over FIGLUT [42]. The main contributions of this work include:

- We propose WAQ LUT-GEMM, a LUT-based GEMM method enabling efficient computation between non-uniformly quantized weights and activations without dequantization. It uses pre-computed Cartesian Product LUTs to significantly reduce LUT size and improve parallelism.
- We introduce an outlier-aware quantization scheme with look-ahead computation and error compensation to efficiently handle activation outliers during inference.
- We propose the architecture design of the OASIS accelerator, which efficiently supports WAQ LUT-GEMM.
- We develop *Orizuru*, a lightweight top-$k$ engine for identifying activation outliers in real-time data streams.

## II. Backgrounds and Motivations

### A. LLM Quantization

LLMs typically utilize FP16 format for weights and activations during inference [1], [10]. Quantization, which reduces the numerical precision of weights and activations, has emerged as a promising technique to optimize LLM inference efficiency [11], [16], [19], [21], [29]. WOQ methods effectively reduce model memory footprint by quantizing the weights [11], [29], but fail to leverage emerging efficient low-precision compute units (e.g., GPUs' INT4 Tensor Cores [41]) since the activations remain in FP16 format. In contrast, WAQ methods quantize both weights and activations, enabling low-precision computation for faster inference [3], [33]. Therefore, recent studies have increasingly focused on developing WAQ methods to further enhance LLM inference efficiency [3], [28], [33], [50], [62].

Conventional WAQ methods quantize both weights and activations into integer representations, which face significant accuracy degradation in low-precision quantization configurations [3], [33], [57], [62]. Compared to integer quantization, which maps floating-point numbers to equally spaced integer levels, non-uniform quantization employs unequally spaced centroids that better align with the actual distributions of weights and activations, thereby reducing quantization noise [2], [34], [54], [60]. To achieve higher accuracy, recent studies have leveraged non-uniform quantization methods to achieve low-precision LLMs [21], [31], [42].

Non-uniform quantization includes low-precision floating-point formats (e.g., MXFP4 [54], NVFP4 [2]) and learned-codebook methods [34], [60]. The learned-codebook methods, which optimize centroids via training, generally achieve higher accuracy [19], [21]. These methods represent data as integer index matrices that reference a floating-point codebook. For example, K-Means quantization [34] can be written as:

$$\tilde{x}_i = C_{idx_i}, \qquad idx_i = \arg\min_k \|x_i - C_k\|^2, \tag{1}$$

where $x_i$ is the original data point, $\tilde{x}_i$ is its quantized value, $C_k$ is the centroid of the $k^{th}$ cluster, and $idx_i$ is the integer index corresponding to $\tilde{x}_i$ that selects the nearest centroid. To represent a matrix of size $M \times N$ with $n$-bit K-Means quantization, an $n$-bit integer index matrix $idx$ of size $M \times N$ and an FP centroid codebook $C$ of size $2^n$ are required.

Although learned-codebook schemes reduce quantization error, they incur substantial runtime cost at inference because current accelerators do not natively support such non-uniform data representations [8], [40]. Performing GEMMs with index-coded data therefore requires per-element codebook lookups and dequantization into FP16 values, followed by FP16 GEMMs, which adds significant overhead [19], [21]. Therefore, unleashing the performance benefits of learned-codebook quantization motivates the design of hardware that directly supports their non-uniform data representations.

### B. LUT-Based GEMM Schemes

LUT-based computation has been demonstrated to be an effective approach for performing efficient GEMMs without

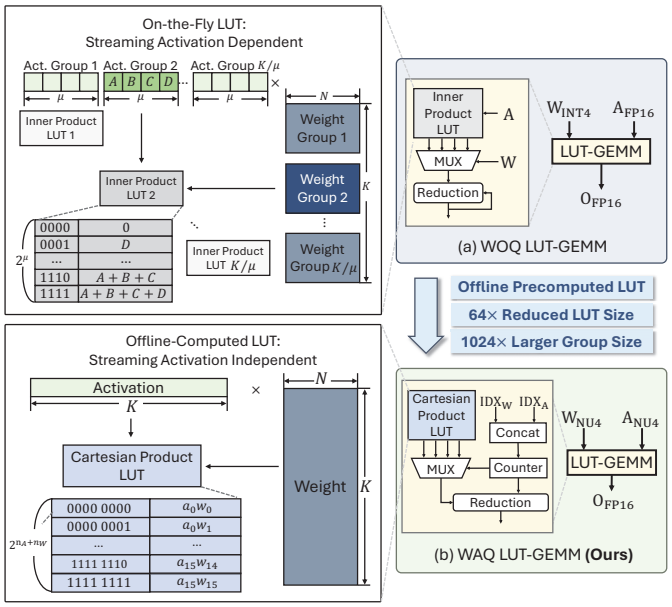


Fig. 2. (a) Existing WOQ LUT-GEMM scheme. $A$, $B$, $C$, and $D$ denote different streaming activation values. (b) Our proposed WAQ LUT-GEMM scheme. $a_i$ and $w_i$ denote the activation and weight centroids, respectively.

TABLE I
COMPARISON OF LUT-BASED GEMM SCHEMES.

| | WOQ LUT-GEMM | Ours |
|---|---|---|
| Wgt. Precision ($n_W$) | NU4 | NU4 |
| Act. Precision ($n_A$) | FP16 | NU4 |
| Offline-Computed LUT? | ✗ | ✓ |
| Group Size | $\mu$ | $K$ |
| LUT Size | $2^\mu \cdot \frac{K}{\mu}$ | $2^{n_A+n_W}$ |
| #FLOPs for Reduction | $\frac{K}{\mu} \cdot n_W \cdot N$ | $2^{n_A+n_W} \cdot N$ |

dequantization cost [37], [42], [43]. Generally, LUT-based GEMM methods take two inputs: which kind of multiplication result is stored in the LUT, and how the LUT is indexed. As shown in Fig. 2(a) and Table I, existing LUT-GEMM methods for WOQ execution [37], [42], [43] store the group-wise inner product results between weights and activations in the LUT and use the weights as the MUX select signals. However, they remain compute-inefficient due to the following limitations: (1) The LUT depends on the streaming activations and therefore must be generated on-the-fly; (2) Let $K$ denote the reduction length of the inner product, a LUT with the size of $2^K$ is required to store all possible inner product results between weights and FP16 activations, which is impractical for large reduction lengths (e.g., $K = 4096$ in LLaMA-7B [52]). To limit the LUT size, existing LUT-based methods partition the weights and activations into small groups of size $\mu$ and perform multiple partial inner products. (3) Partial-sum reductions are required across multiple groups, incurring additional FLOPs and latency. Existing WOQ LUT-GEMM methods adopt bit-serial weight processing to further reduce the LUT size, with each cycle only processing one bit of the weights [37], [42], [43].

To balance between the LUT size and computational parallelism, weights and activations are grouped into groups of size $\mu = 4$ [37], [42]. Among the existing WOQ LUT-GEMM methods, FIGLUT [42] and LUT Tensor Core [37] manage to reduce the LUT size by half by using the most-significant-bit (MSB) of the weight index as the enable signal of the negation logic. However, the LUT sizes of these methods are still large, and the computational parallelism remains limited.

While WOQ LUT-GEMM methods can be adapted to avoid dequantizing weights and activations in NU-WAQ GEMMs, we observe three opportunities specific to NU-WAQ that further improve the compute efficiency: (1) In NU-WAQ, activation centroids are trained offline, so the set of possible activation values at runtime is known in advance, allowing the LUT to be precomputed and stored on-chip prior to inference. (2) With both weights and activations quantized, the number of distinct Cartesian Product results is limited; storing the Cartesian Product of weight and activation centroids in the LUT instead of the inner product results substantially reduces LUT sizes. (3) A Cartesian Product LUT is independent of the reduction length, allowing computation at a larger granularity and significantly eliminating the number of FLOPs during reductions.

Leveraging the aforementioned opportunities, we propose a novel LUT-based GEMM scheme for NU-WAQ, as illustrated in Fig. 2(b). The inputs of our design are weights and activations that are non-uniformly quantized using learned codebooks. The LUT stores the Cartesian Product of weight and activation centroids, which can be precomputed and loaded on-chip before inference. At runtime, weight and activation indices are concatenated. The occurrence counts of the unique concatenated indices are calculated and used to perform reductions as weighted sums of the corresponding LUT entries. These unique concatenated indices act as MUX select signals to fetch the corresponding Cartesian Product values from the LUT.

The configuration comparison between existing WOQ LUT-GEMM methods and our proposed WAQ LUT-GEMM method is summarized in Table I. $n_W$ and $n_A$ denote the bit-widths of weights and activations, respectively. Consider an $M-K-N$ GEMM example between weights and activations where $M = 1$ and $N = K = 4096$, which is a common case in the LLaMA-7B model [52]. For the configuration of $n_W = n_A = 4$, our proposed WAQ LUT-GEMM method achieves a 64× reduction in LUT size, a 1024× increase in group size, and a 16× reduction in floating-point operations (FLOPs) for reductions over existing WOQ LUT-GEMM methods [37], [42], [43].

### C. Handling Activation Outliers in WAQ

In LLMs, activation outliers exhibit both higher quantity and magnitude compared to weight outliers, which expands the quantization range and reduces the effective bit resolution available for the majority of values (inliers) [33]. To mitigate activation outlier-induced quantization errors, prior works have

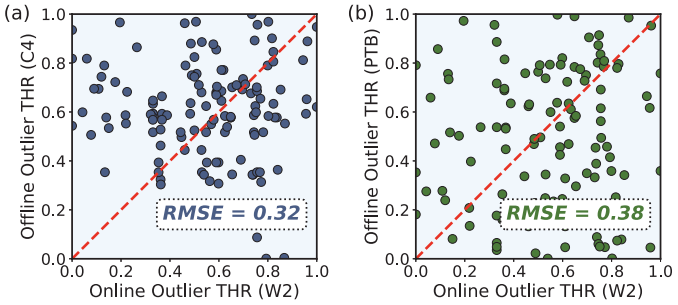


Fig. 3. Comparison between online and offline derived upper activation outlier thresholds. The online dataset is WikiText-2 [36] (W2), and the offline dataset is (a) C4 [9] and (b) PTB [35]. The activations used to compute the thresholds and centroids are the input of the $1^{st}$ $q\_proj$ layer in the LLaMA-3-8B model [15]. 128 activation tokens are applied to compute the thresholds, each with the dimension of $1 \times 4096$. The thresholds are normalized to [0, 1].

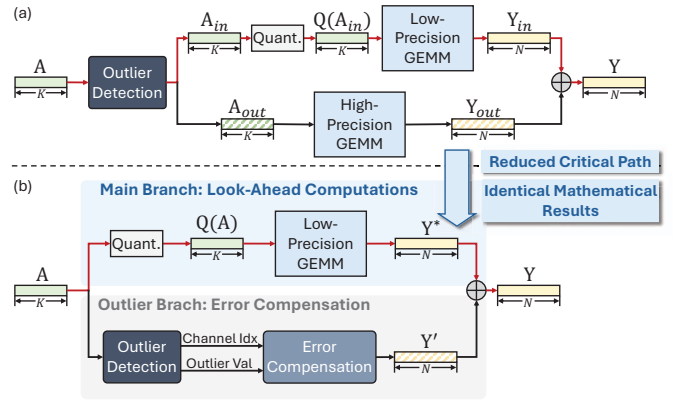


Fig. 4. Comparison of (a) conventional dynamic outlier detection and (b) the proposed look-ahead scheme.

proposed to isolate outliers and preserve them in higher-precision representations (e.g., INT8 or FP16) during computation [19], [62]. Existing works basically identify outliers with two approaches: (1) dynamically identify top-k outliers during inference [19]; and (2) leveraging an offline calibration dataset to identify certain activation channels that contain a large volume of outliers as outlier channels. During online inference, the channel indices are used to indicate which activation values are outliers [32], [62]. It has been evaluated that dynamic outlier detection generally yields higher accuracy than static outlier channel identification [19]. To explain this, we define the value of the top-k largest activation as the upper outlier threshold.

Fig. 3 demonstrates substantial discrepancies in upper outlier thresholds between offline and online datasets. The online calibration uses WikiText-2 [36], while online inference uses C4 [9] and PTB [35] in Fig. 3(a) and (b), respectively. The root mean square error (RMSE) between online and offline thresholds is 0.32 and 0.38 in Fig. 3(a) and (b), respectively, indicating low similarity.

Therefore, to preserve model accuracy, it is crucial to dynamically identify outliers during inference.

Fig. 4(a) illustrates the conventional workflow of dynamic outlier detection during inference [19]. Generally, the entire activation vector is scanned to identify outliers, separating the

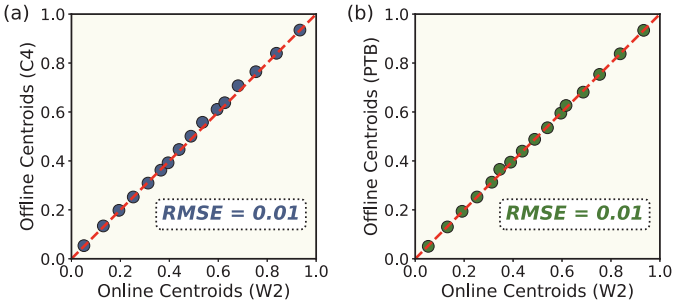


Fig. 5. Comparison between online and offline derived 4-bit quantization centroids for activations. The configurations of activations are the same as those in Fig. 3. The centroids are normalized to [0, 1].

activations into two groups: inliers ($A_{in}$), which are quantized into low-precision formats; and outliers ($A_{out}$), which are retained in higher-precision formats. The activation inliers and outliers undergo low-precision GEMMs and high-precision GEMMs with the weights, respectively, and their results $Y_{in}$ and $Y_{out}$ are combined to produce the final output. While the GEMMs associated with inliers and outliers can be executed in parallel on separate compute units, the outlier detection process stays on the ***critical path*** (indicated with red arrows), incurring significant latency overhead.

To address this issue, we propose a look-ahead computation scheme, as illustrated in Fig. 4(b). To avoid introducing additional runtime overhead for handling outliers, we propose two concurrent compute branches: the *main branch* performs ***look-ahead*** WAQ-LUT GEMMs with the entire activation quantized, ignoring the quantization errors from outliers; meanwhile, the *outlier branch* calculates the quantization errors from outliers and performs ***error compensation***. The final GEMM result is obtained by summing the outputs of look-ahead GEMMs ($Y^*$) and error compensation ($Y'$), which leads to identical mathematical results as conventional dynamic outlier detection, but without incurring additional latency.

### D. Our Approach

We aim to achieve efficient LLM inference with high model performance by leveraging NU-WAQ. Driven by the aforementioned observations and analysis, in this paper, we propose OASIS, an algorithm-architecture co-design framework. Specifically, § III details the computation optimizations of OASIS, which focus on the following two aspects: (1) to enable efficient GEMMs between the inliers of non-uniformly quantized weights and activations, we propose a WAQ LUT-GEMM scheme without requiring dequantization; and (2) we introduce an outlier-aware GEMM computation scheme that concurrently performs computations on both activation inliers and outliers. We also present the architecture design of the OASIS accelerator in § IV, which efficiently implements the proposed computation schemes.

## III. Computation Optimizations

In this section, we present the computation optimizations to enable efficient LLM inference with NU-WAQ. Specifically, § III-A describes the quantization method used in OASIS,

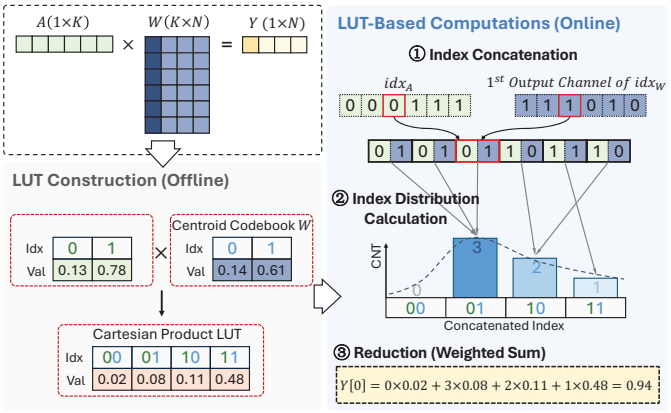


Fig. 6. WAQ LUT-based GEMM computation scheme.

§ III-B presents the WAQ LUT-based GEMM scheme for efficient GEMMs with non-uniformly quantized weights and activations, and § III-C introduces the look-ahead computation and error compensation designs to handle activation outliers.

### A. Quantizing Weights and Activations

To maintain model performance in low-precision configurations, OASIS adopts the learned-codebook quantization method of K-Means [34] for weight and activation quantization. Specifically, we employ output-channel-wise quantization for weights and token-wise quantization for activations. For weights, the entire weight matrix shares the same quantization centroids, while each output channel has its own scaling factor. For activations, each token has its own set of quantization centroids and scaling factors. We quantize the pretrained LLM weights to obtain the weight centroids, while the activation centroids are learned through an offline calibration dataset. As shown in Fig. 5, the offline and online activation centroids exhibit high consistency across different dataset configurations. The RMSE values between the offline and online centroids are both only 0.01 in Fig. 5(a) and (b). This indicates the feasibility of using offline-learned activation centroids for online activation quantization to avoid the overhead of activation centroid learning during inference. To further mitigate model performance degradation caused by activation quantization, we dynamically identify the top-0.5% largest and the bottom-0.5% smallest of the activations as outliers and preserve them in FP16 format.

### B. WAQ LUT-Based GEMM

Leveraging the WAQ-specific opportunities discussed in Section II-B, we propose a WAQ LUT-based GEMM scheme to efficiently execute GEMMs between non-uniformly quantized weights and activations. Fig. 6 shows an example of computing the GEMM output with the activation and the first output channel of weights using the proposed WAQ LUT-based GEMM scheme. In this $M-K-N$ GEMM example, $M=1, K=6, N=4$, and $n_W = n_A = 1$.

In learned-codebook WAQ methods, both the quantization centroids of weights and activations are determined offline. In

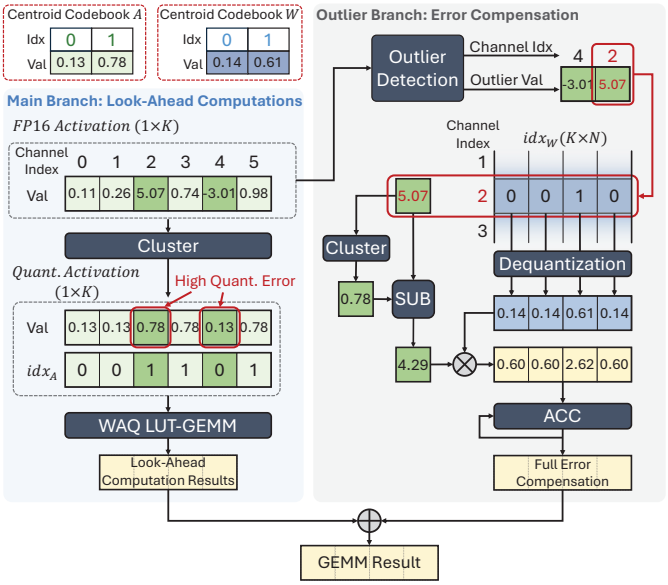


Fig. 7. Look-ahead computations and error compensation.

other words, unique weight and activation values are predefined before inference. Therefore, we construct the Cartesian Product LUT offline, which stores all possible multiplication results between the weight and activation centroids. During online inference, we concatenate the indices of the activations and weights, which is shown in step ①. Then, in step ②, we calculate the distribution of these concatenated indices. Finally, in step ③, we perform the reduction of the multiplication results along the input channel dimension ($K$). Specifically, we replace the $K$ FP16 additions in the conventional GEMM with a weighted sum of the multiplication results stored in the Cartesian Product LUT. The counts of each unique concatenated index serve as weights of the weighted sum. The number of FP16 additions is reduced from $K$ to $2^{n_W+n_A}$, which is significantly smaller in low-precision quantization scenarios.

Consider the common case of W4A4 GEMMs, where $n_W = n_A = 4$. The Cartesian product LUT stores $2^{n_W+n_A}$ multiplication results—only 256 entries. This LUT size is 64× smaller than the inner-product LUT used in existing WOQ LUT-GEMM methods for a 4096×4096 weight layer. Therefore, unlike WOQ LUT-GEMM methods that require small group sizes to control LUT size, our design can support arbitrary reduction lengths without increasing LUT size, enabling higher parallelism. In our WAQ LUT-GEMM scheme, the reduction length is equal to the input channel number of the weights $K$. The high computational parallelism of our design yields a 16× reduction in FLOPs compared to existing WOQ LUT-GEMM methods. These advantages become more pronounced for larger LLMs as per-layer input channel numbers increase [10], which is evaluated in §V-D.

### C. Look-Ahead Computations and Error Compensation

As discussed in §II-C, it is important to effectively hide the latency of outlier detection. To address this challenge, we propose a look-ahead computation and error compensation design

that concurrently performs WAQ LUT-GEMM computations and outlier detection. Fig. 7 shows an example of the proposed look-ahead computation and error compensation design, which are performed in two parallel branches: the main branch (left) and the outlier branch (right). In this $M-K-N$ GEMM example, $M=1$, $K=6$, $N=4$, and $n_W=n_A=1$. There are two activation outliers in channels 2 and 4, indicated by deep green boxes.

*1) Look-Ahead Computations:* In the main branch, the entire FP16 activation vector is clustered to the nearest centroids in the activation codebook $\mathcal{C}_A$, producing quantized activations. While the quantization error of the inliers is generally small and does not lead to significant accuracy degradation, the quantization error of the outliers is non-negligible. For instance, the activation outliers in channels 2 and 4, originally $5.07$ and $-3.01$, are quantized to $0.78$ and $0.13$, yielding quantization errors (residuals) of $4.29$ and $-3.14$, respectively. Our proposed look-ahead computation design allows the main branch to temporarily ignore the quantization errors of outliers and directly perform WAQ LUT-GEMM computations using the quantized activations, while the outlier branch simultaneously computes the error compensation for the outliers.

*2) Error Compensation:* In the outlier branch, the outlier detection units dynamically identify the outliers in the activation vector and compute their quantization errors by subtracting the quantized activation outliers from the original FP16 activation outliers. We use the channel index of the activation outliers to fetch the corresponding input channels of quantized weights from the $idx_W$ matrix, which are then dequantized to FP16 format based on the weight codebook $\mathcal{C}_W$. Then, the residuals are multiplied by the dequantized weights to generate error compensation terms, which are accumulated into the look-ahead computation results using WAQ LUT-GEMM from the main branch. As shown in Fig. 4, this mechanism removes the time-consuming dynamic outlier detection from the critical path, enabling reduced GEMM runtime and ensuring identical mathematical results.

Note that in each cycle, the outlier detection unit sequentially outputs each pair of outlier value and their channel index. Thus, in each cycle, only one input channel of the weight index matrix is fetched and dequantized for error compensation. Compared to the conventional design for dynamic outlier identification (Fig. 4(a)) which performs sparse FP16 GEMMs for all residuals simultaneously, this design eliminates the need for sparse representation of the outliers and significantly reduces the number of multiply-accumulate (MAC) units required in the outlier branch.

## IV. Hardware Design

To enable the efficient execution of the computation optimizations introduced in §III, in this section, we present the supporting accelerator design of OASIS. First, we present the overall architecture of the OASIS accelerator. Then, we describe the micro-architecture designs of the Index Counter and the Clustering Unit, which are the key hardware components to support the proposed WAQ LUT-based GEMM. Moreover, we also propose a lightweight outlier detection engine *Orizuru*, which dynamically identifies the top-$k$ largest and smallest elements in each activation token with minimal overhead.

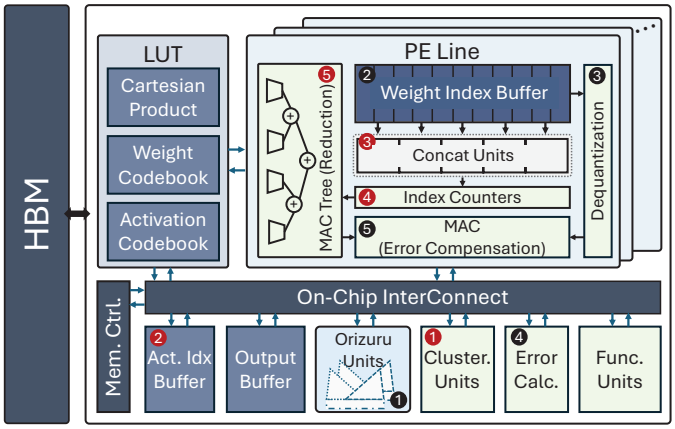

Fig. 8. Overall architecture of the OASIS accelerator.

### A. Overall Architecture

Fig. 8 shows the overall architecture of the OASIS accelerator, and Table II presents its hardware configurations. The OASIS accelerator is composed of 16 Processing Element (PE) Lines, a LUT, an Activation Index Buffer, an Output Buffer, *Orizuru* units, Clustering Units, Functional Units, an Error Calculation Unit, and a Memory Controller. Besides the Cartesian Product results, the LUT also stores the centroid codebooks of activations and weights for the purposes of activation clustering and weight dequantization, respectively. The main compute unit within the PE Lines is the Concat Unit, which accepts two 4-bit indices, concatenates them, and stores the result in an 8-bit register. The Concat Unit's minimalist design provides high area efficiency compared to FP16 MAC units, benefiting both compute-intensive and memory-intensive scenarios. For compute-intensive workloads such as LLM prefill phase, this efficiency enables a higher number of Concat Units on the chip, delivering the computational parallelism needed for high throughput. For memory-intensive workloads such as LLM decode phase, the lightweight Concat Units consume minimal chip area, freeing up chip area for additional I/O pins to increase bandwidth and mitigate memory bottlenecks.

Within each PE Line, there are 4096 Concat Units, a Weight Index Buffer, 32 Index Counters, a Dequantization Unit, a MAC Tree for reduction, and 8 MAC Units for error compensation. The computation flow of the main and outlier branches is illustrated with the red and black circled numbers, respectively.

For the main branch, in step ❶, the Clustering Units reads the entire activation vector in the Output buffer, and clusters them to the nearest centroids based on the Activation Codebook. The clustered activation indices are then stored in the Activation Index Buffer. In step ❷, the clustered activation indices are fetched from the Activation Index Buffer and broadcast to all PE Lines. In step ❸, the Concat Units in each PE Line concatenate the activation indices with the weight

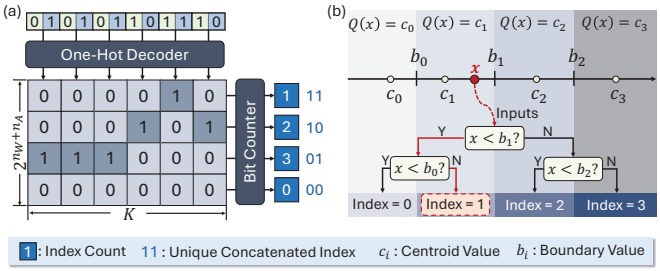


Fig. 9. Design of (a) the Index Counter and (b) the Clustering Unit.

indices fetched from the Weight Index Buffer to form the concatenated indices. Next, in step ❹, the Index Counters calculate the counts of the unique concatenated indices. Finally, in step ❺, the MAC Tree performs the weighted-sum operations based on the counts and the corresponding Cartesian Product values retrieved from the LUT, generating the look-ahead computation results.

For the outlier branch, in step ❶, the *Orizuru* units read the FP16 activations in the Output buffer and dynamically identify the outliers, sequentially outputting each outlier value along with its channel index. The weight input channel corresponding to the outlier channel index is fetched from the Weight Index Buffer in step ❷, and dequantized to FP16 weights in the Dequantization Unit in step ❸. In step ❹, the Error Calculation Unit calculates the residual between the outlier activation and its nearest centroid from the Activation Codebook. Step ❹ is performed in parallel with ❷ and ❸. Then, in step ❺, the MAC Units perform multiply-accumulates between the outlier residuals and the dequantized weights, generating the error compensation terms. Finally, the MAC units combine the error compensation terms with the look-ahead computation results from the main branch to produce the final output activations, which are stored back in the Output Buffer.

To minimize end-to-end latency, the Memory Controller orchestrates pipelined execution of operations across both the main and outlier branches. The cycle latencies for each computation step are presented in § V-D3.

### B. Index Counter

In OASIS, we design an Index Counter to compute the index distribution of the concatenated indices with high parallelism, which is illustrated in Fig. 9(a). The concatenated indices are decoded into one-hot vectors in parallel, and the bit counters calculate the row-wise sums of the one-hot vectors to obtain the count for a certain concatenated index. For example, the first concatenated index '01' is decoded into the one-hot vector [0, 0, 1, 0] as shown in the first column of the decoded one-hot matrix in Fig. 9(a). In the first row of the decoded one-hot matrix, there is one '1', indicating that the concatenated index '11' appears once in the concatenated indices. The row-wise sums of the second, third, and fourth rows correspond to the counts of concatenated indices '10', '01', and '00', respectively. To satisfy timing and area constraints, the Index Counter employs a 16-input design, with each PE Line incorporating 32 Index Counters to perform the index distribution calculations in parallel.

TABLE II
OASIS ACCELERATOR CONFIGURATIONS (28NM, 500MHZ).

| Module | | Specification | Area ($mm^2$) | Power ($W$) |
|---|---|---|---|---|
| PE Line | | 16 PE Lines per chip | 9.08 | 7.54 |
| PE Line | Concat Unit | 4096 Concat Units per line | $8.68 \times 10^{-2}$ | $8.36 \times 10^{-2}$ |
| | Wgt Idx Buffer | 2 KB per line | $6.75 \times 10^{-2}$ | $1.69 \times 10^{-2}$ |
| | Index Counter | 32 16-in Index Counters per line | $2.71 \times 10^{-1}$ | $6.14 \times 10^{-2}$ |
| | Dequant Unit | 1 Dequant Unit per line | $2.83 \times 10^{-3}$ | $6.11 \times 10^{-3}$ |
| | MAC Tree | 1 32-in FP16 MAC Tree per line | $1.17 \times 10^{-1}$ | $2.54 \times 10^{-1}$ |
| | MAC | 8 FP16 MAC Units per line | $2.26 \times 10^{-2}$ | $4.89 \times 10^{-2}$ |
| Output Buffer | | 64 KB per chip | 2.17 | $2.68 \times 10^{-1}$ |
| Act. Idx Buffer | | 16 KB per chip | $5.40 \times 10^{-1}$ | $6.71 \times 10^{-2}$ |
| LUT | | 2 KB per chip | $6.75 \times 10^{-2}$ | $8.38 \times 10^{-3}$ |
| Cluster. Unit | | 4 Clustering Units per chip | $1.31 \times 10^{-3}$ | $2.90 \times 10^{-4}$ |
| Orizuru | | 273 16-in Orizuru Units per chip | $7.39 \times 10^{-1}$ | $2.73 \times 10^{-1}$ |
| Error Calc. Unit | | 1 Error Calculation Unit per chip | $4.12 \times 10^{-3}$ | $6.40 \times 10^{-3}$ |
| Func. Unit | | 1 Functional Unit per chip | $8.89 \times 10^{-1}$ | $5.63 \times 10^{-1}$ |
| Memory Controller | | 1 Memory Controller per chip | 1.47 | $9.28 \times 10^{-1}$ |
| Total | | — | 15.31 | 9.66 |

### C. Clustering Unit

To perform efficient non-uniform quantization for the activations, we design a Clustering Unit as illustrated in Fig. 9(b), which maps each activation to its nearest centroid based on the pre-defined codebook. The clustering unit first computes the boundary values between each pair of adjacent centroids: $b_i = (c_i + c_{i+1})/2$, where $c_i$ and $c_{i+1}$ are two adjacent centroids. For any input value which is within $[b_{i-1}, b_i)$, it is assigned to the $i$-th cluster with centroid $c_i$. Then, for each input activation $x$, the clustering unit compares it with the boundary values to determine the cluster index it belongs to. To accelerate the clustering process, we implement the comparison logic using a binary search tree structure. In the example of Fig. 9(b), there are 4 centroids, and each input activation $x$ undergoes $log_2(4) = 2$ hierarchical comparisons to determine its cluster index.

### D. Orizuru: Dynamic Outlier Detection Engine

As mentioned in § III-C, OASIS minimizes runtime by performing outlier detection on a dedicated outlier branch that runs in parallel with the main branch. To avoid making the outlier branch the performance bottleneck of the entire GEMM operation, we design *Orizuru* [1], an efficient outlier detection engine that identifies the outliers from each activation token with minimal latency and energy consumption.

We propose to detect the top $k$ largest and smallest values within each activation token $\mathbf{x} = [x_i] \in \mathbb{R}^N$, which are excluded from the quantization process. To minimize the number of comparisons required for outlier detection, we adopt a tree-based architecture which maximizes the reuse of comparison results. As shown in Fig. 10(a), *Orizuru* is composed of two complete binary trees with shared leaf nodes. These two binary trees, named the max tree, $\mathcal{P}$, and the min tree, $\mathcal{Q}$, are used

[1]The shape of the two-fold binary trees with shared leaves resembles an *Orizuru* (a paper crane).

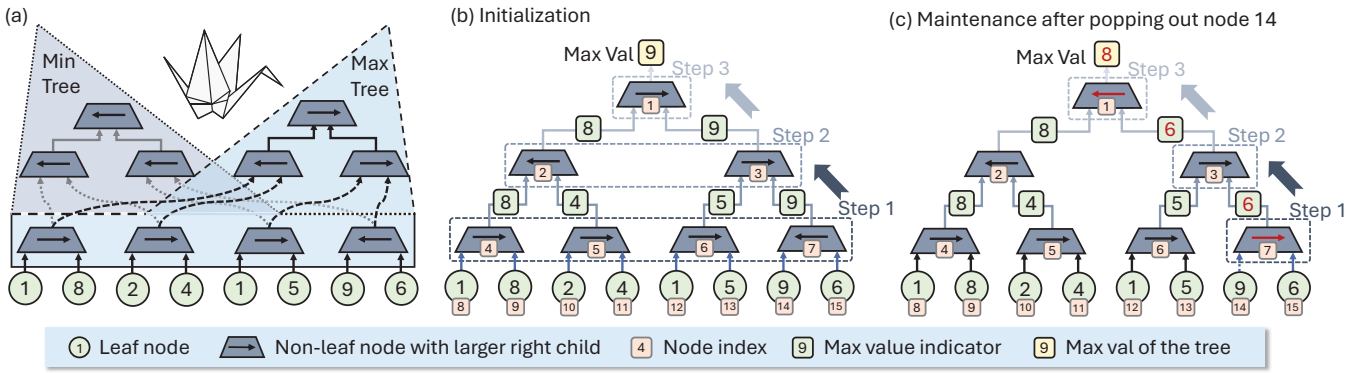


Fig. 10. Orizuru architecture. (a) The overall architecture of Orizuru: two-fold binary trees with shared leaf nodes and comparison results at the last layer of the non-leaf nodes. (b) Initialization of the max tree. (c) Maintenance of the max tree after popping out node 14.

to pop the $k$ largest and smallest elements of a given vector $\mathbf{x}$, respectively. The tree structure offers high data reuse for input data, reducing memory access overhead. Without loss of generality, we take the max tree as an illustration of how we initialize, pop out, and maintain the tree; then we showcase how we reuse the information from the max tree to the min tree towards the *Orizuru* architecture.

**Complete binary tree architecture** For simplicity, in the example of Fig. 10, we consider an 8-input *Orizuru* for $N = 8$. Specifically for the max tree, $\mathcal{P}$, as shown in Fig. 10(b), we build a complete binary tree with $L$ levels, where $L = \log_2(N)$. Each node on the tree is a 2-to-1 multiplexer (MUX) controlled by the value in a bit buffer, denoted as $p_{l,i} \in [0, 1]$, where $l = 1, 2, \ldots, L$ is the level index and $i = 1, 2, \ldots, 2^{l-1}$ is the node index of this level. For the $N/2$ leaf nodes on the $L$-th level, the node $p_{L,i}$ is directly connected to the FP16 values of $x_{2i-1}$ and $x_{2i}$; the non-leaf node $p_{l,i}$ is connected to its left/right children $p_{l+1,2i-1}$ and $p_{l+1,2i}$, and its MUX outputs

$$\mathrm{MUX}(p_{l,i}) = \begin{cases} \mathrm{MUX}(p_{l+1,2i-1}) \text{ or } x_{2i-1}, & p_{l,i} = 0 \\ \mathrm{MUX}(p_{l+1,2i}) \text{ or } x_{2i}, & p_{l,i} = 1. \end{cases} \quad (2)$$

In the max tree, $\mathcal{P}$, the bit buffer points to the larger child node or activation value, which represents the largest element in the sub-tree rooted at the current node. Consequently, the output of the root node, $\mathsf{MUX}(1, 1)$, corresponds to the largest value in the entire activation vector, $\max(\mathbf{x})$. To track the availability of activation elements, we introduce a mask vector $\mathbf{m}^{(p)} = [m_i^{(p)}] \in [0, 1]^N$ for the max tree $\mathcal{P}$, where $m_i^{(p)} = 1$ indicates that the activation element $x_i$ is available, and $m_i^{(p)} = 0$ indicates it has already been popped out. Similarly, another independent mask vector $\mathbf{m}^{(q)} = [m_i^{(q)}] \in [0, 1]^N$ is defined for the min tree $\mathcal{Q}$.

**Initializing the binary tree** To initialize the max and min trees for a new activation vector $\mathbf{x}$, we update all the bit buffers tree, which is shown in Fig. 10(b) for the max tree. This process is performed in a bottom-up manner across the $L$ levels of the tree. Initially, at the $\log_2(N)$-th level, the registers of the non-leaf nodes (e.g., nodes 4, 5, 6, and 7) are updated by comparing their left and right child nodes at the leaf level, which hold the input values. This step involves $N/2$ comparisons. After completing these comparisons, the registers are updated to indicate whether the left or right child contains the larger value. Next, this comparison process is repeated recursively for each level, moving upwards until the root node's register is updated. Finally, the MUX at the root node selects the maximum value from the $N$ elements. Overall, the tree initialization requires $N - 1$ FP16 comparisons.

**Popping out and maintaining the tree** Once the initialization is complete, the maximum value can be popped out immediately. The index of this maximum value is determined by traversing the tree from the root node to the corresponding leaf node, following the binary digits stored in the registers. For example, to locate the index of the maximum element "9" (at node 14) in Fig. 10(b), we start at the root node, which contains "1", directing us to its right child, node 3. Node 3 also contains "1", further directing us to its right child, node 7. This process continues for $\log_2(N)$ steps until we reach the maximum element "9" at node 14. The binary digits from the accessed registers are concatenated to form "110". Since leaf nodes are indexed from $N$ to $2N - 1$, a "1" is prefixed to the concatenated value, resulting in "1110"—the binary representation of the index "14" for the maximum value "9". Note that this process does not involve any comparisons and can be completed in one clock cycle.

To identify the second largest value, the popped maximum value must first be effectively "removed" from the tree, which is the tree maintenance process. This is achieved by updating the registers of the non-leaf nodes while maintaining the tree structure. As shown in Fig. 10(c), the updated information is highlighted with red Max val indicators and arrows. Specifically, the popped maximum value is treated as negative infinity during the update for the register value of its parent node, while the original value remains unchanged in the FP16 buffer of the corresponding leaf node. Consequently, the arrow in the parent node flips to point to the neighbor node of the popped value.

The registers of the ancestors of the popped element are updated in a bottom-up manner. This maintenance process resembles the initialization but involves only a single comparison per level. Each maintenance step requires $\log_2(N)$ sequential comparisons. To retrieve the top-$k$ maximum values, this popping and maintenance process is repeated $k$ times.

Note that after repeating the maintenance process multiple times, there may be cases where both leaf nodes under a certain non-leaf node are popped out. For instance, after retrieving the top-3 largest values, both child nodes of node 7 may be popped out. In this case, the MUX of node 7 outputs negative infinity for the register update on its parent node (node 3) in the subsequent cycle. Obviously, as long as the tree is not entirely emptied, i.e., fewer than $N$ elements have been popped out, this negative infinity cannot propagate all the way to the root node.

**Combining two trees into *Orizuru*** A key advantage of *Orizuru*'s two-fold binary tree architecture is its ability to reuse the max tree's results to reduce the number of comparisons needed for the min tree. Given the limited number of comparators, the runtime bottleneck in *Orizuru* occurs during the initial tree setup, which requires $N/2$ comparisons. However, the comparison results from this step can be directly reused to initialize the min tree. Specifically, the registers at the $\log_2(N)$-th level of the min tree can adopt the reversed comparison results from the max tree. This allows the min tree's initialization to skip the $\log_2(N)$-th level and begin directly at the $(\log_2 N - 1)$-th level, reducing the total comparisons for its initialization by 50%.

In summary, *Orizuru* picks the $k$ maximums and $k$ minimums from an $N$-input activation vector $\mathbf{x}$ at the cost of $1.5N + 2k \cdot \log_2(N)$ comparisons. This is significantly smaller than the $6N$ comparisons required by the top-$k$ engine in [55].

**Dealing with ties** Since the outlier detection is performed on the FP16 activations, which have limited precision, multiple activation values within each token can be identical, leading to ties when determining the $k$-th largest or smallest values. On average, such ties occur in approximately 2% of activation tokens across all evaluated layers and models. To address this issue, we always output exactly $k$ outliers for each of the max and min trees to maintain a consistent number of outliers for error compensation. This is achieved by the following tie-breaking strategy: in cases where there are ties in the comparison results, we deterministically select the left child node as the larger value in the max tree and as the smaller value in the min tree.

## V. Evaluation

### A. Experimental Setup

**Models and Tasks** We examine the algorithm performance of OASIS on a spectrum of LLMs and tasks. Specifically, the models include OPT-6.7B/13B/30B [61], LLaMA-7B/13B/30B [52], LLaMA-2-7B/13B/70B [53], LLaMA-3-8B [15], and Mistral-7B [20], which are implemented with Transformers [56] and PyTorch [44]. These LLMs are evaluated on two tasks: (i) the next-word prediction task using the WikiText-2 [36] dataset, measured by the perplexity (PPL) metric, and (ii) the zero-shot accuracy task across six common sense datasets: PIQA [4], ARC-easy (ARC-E) [7], ARC-challenge (ARC-C) [7], BoolQ [6], HellaSwag [59], and WinoGrande [47]. The zero-shot performance evaluation utilizes the Language Model Evaluation Harness [13] framework. All algorithm performance experiments are conducted on an NVIDIA A100-80GB GPU.

**OASIS' NU-WAQ Implementation Details** In OASIS, both weights and activations are quantized using K-Means clustering. Weights undergo 4-bit per-output-channel quantization without outlier protection, while activations are quantized per-token with 3/4-bit precision. We first perform post-training K-Means quantization on the LLM weights to obtain the weight centroids and indices. Next, the activation centroids are trained offline using 16 calibration samples from the C4 dataset [9], and the indices are computed online. We incorporate a weighted-K-Means algorithm to obtain the activation centroids, where the weights are determined by Fisher information matrices [45] of the activations. To handle outliers, the top 0.5% largest and bottom 0.5% smallest activation values are preserved in FP16 format, while the inliers are quantized. During inference, OASIS dynamically identifies outliers using the *Orizuru* units, while OASIS-S reuses the thresholds from the offline training process on the calibration dataset.

**Baseline LLM Quantization Methods** We compare OASIS with INT-WAQ baselines round-to-nearest (RTN), SmoothQuant [58], QuaRot [3], and Atom [62]. Except for Atom, which uses group-wise quantization for both weights and activations with a group size of 128, all other baseline algorithms employ per-output-channel quantization for weights and per-token quantization for activations.

**Architecture Modeling and Comparison** The hardware performance of the OASIS architecture is modeled using a cycle-accurate simulator modified from DnnWeaver [48]. The area and power metrics of the core logic units in the OASIS accelerator, such as the Concat Unit, MAC Tree, Index Counter, and *Orizuru*, are derived from synthesis results using the TSMC 28 nm standard cell library. Table II shows the detailed configurations of the hardware components on-chip.

We use Cacti [27] and DRAMSim3 [26] to simulate the overhead of on-chip SRAM and off-chip HBM, respectively. We denote W4A3 OASIS as OASIS-A3 and W4A4 OASIS as OASIS-A4, and similarly for OASIS-S. We compare the hardware performance of OASIS with a series of baseline hardware accelerators, including the GPU-based platforms of NVIDIA A100-80GB GPU [40] and QuaRot [3] and an ASIC accelerator of FIGLUT [42]. Unless otherwise specified, the hardware performance of OASIS and the baseline accelerators are evaluated on the next-word-prediction task with an output sequence length of 2048.

### B. Algorithm Performance Analysis

Table III shows the WikiText-2 PPL results for OASIS and baseline INT-WAQ methods across various models with a sequence length of 2048. OASIS consistently achieves the lowest PPL for both W4A4 and W4A3 precisions, outperforming the INT-WAQ methods. This demonstrates the effectiveness of OASIS's NU-WAQ and outlier protection methods in reducing quantization errors and enhancing model performance. For

TABLE III
WIKITEXT2 PERPLEXITY WITH 2048 SEQUENCE LENGTH.

| Precision | Method | OPT | | | LLaMA | | | LLaMA-2 | | | LLaMA-3-8B | Mistral-7B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 6.7B | 13B | 30B | 7B | 13B | 30B | 7B | 13B | 70B | | |
| **FP16** | - | 10.86 | 10.12 | 9.56 | 5.68 | 5.09 | 4.10 | 5.47 | 4.88 | 3.32 | 6.14 | 5.25 |
| **W4A4** | RTN | 6e3 | 3e4 | 7e3 | 8e3 | 1e4 | 3e5 | 2e3 | 7e3 | 2e5 | 2e3 | 6e3 |
| | SmoothQuant | 2e4 | 7e3 | 1e4 | 4e2 | 67.20 | 32.51 | 7e2 | 56.61 | 10.54 | 1e3 | 5e2 |
| | QuaRot | 12.21 | 11.20 | 10.92 | 6.34 | 5.58 | 4.64 | 6.19 | 5.45 | 3.83 | 8.16 | 5.77 |
| | Atom† | 12.05 | 10.99 | 10.74 | 6.25 | 5.52 | 4.61 | 6.12 | 5.31 | 3.73 | 8.10 | 5.76 |
| | OASIS-S | 11.77 | 10.93 | 10.31 | 6.08 | 5.38 | 4.40 | 6.00 | 5.21 | 3.60 | 7.02 | 5.84 |
| | **OASIS** | **11.62** | **10.75** | **10.21** | **6.04** | **5.37** | **4.38** | **5.90** | **5.19** | **3.55** | **7.11** | **5.75** |
| **W4A3** | RTN | 3e4 | 2e4 | 2e4 | 2e4 | 2e4 | 1e4 | 6e5 | 5e5 | 6e5 | 1e5 | 1e4 |
| | SmoothQuant | 7e4 | 7e4 | 6e4 | 5e4 | 2e4 | 2e4 | 8e3 | 1e4 | 1e4 | 8e3 | 9e3 |
| | QuaRot | 2e2 | 2e2 | 1e2 | 29.75 | 19.02 | 13.50 | 2e2 | 2e2 | 85.28 | 3e2 | 2e2 |
| | Atom† | 20.51 | 15.61 | 14.48 | 9.62 | 7.36 | 6.18 | 11.40 | 8.00 | 5.05 | 13.11 | 10.83 |
| | OASIS-S | 15.12 | 13.49 | 12.14 | 7.60 | 6.28 | 5.31 | 7.91 | 6.99 | 4.13 | 8.96 | 7.42 |
| | **OASIS** | **14.12** | **12.84** | **11.78** | **7.17** | **6.21** | **5.10** | **7.49** | **6.43** | **4.05** | **8.18** | **7.27** |

† Atom applies group quantization to weights and activations, with the group size of 128.

LLaMA-2-7B at W4A4, OASIS achieves a PPL of 5.90, with only a 0.43 degradation from the FP16 model, which is 34% lower than Atom's degradation. Additionally, OASIS reduces PPL by 0.05 at W4A4 and 0.27 at W4A3 compared to OASIS-S, highlighting the benefits of dynamic outlier detection. For the LLaMA-3-8B, which is known to be more quantization-sensitive, OASIS achieves a PPL of 7.11 at W4A4, which reduces the PPL degradation by 49% compared to Atom. We notice that for LLaMA-2-7B and 13B, W4A3 quantization yields higher PPL than their counterparts in the LLaMA-7B and 13B models, because the more extensively trained LLaMA-2 models are harder to post-training quantize at low precisions [22].

On average, OASIS introduces only a 2.05% and 5.90% accuracy drop at W4A4 and W4A3 precision levels, respectively, compared to the FP16 baseline, while significantly outperforming state-of-the-art INT-WAQ methods. In the W4A4 setting, OASIS improves accuracy by 6.44% and 6.92% compared to Atom and QuaRot, respectively. Under the W4A3 configuration, OASIS achieves accuracy improvements of 8.79% over Atom and 30.44% over QuaRot.

### C. Hardware Performance Analysis

For hardware performance, we evaluate OASIS (an NU-WAQ design) against FP16, INT-WAQ, and WOQ LUT accelerators. FP16 inference is run on the A100 GPU. For the INT-WAQ baseline, we deploy QuaRot's W4A4 GEMM kernel on the A100, since Atom's kernel is only available for LLaMA-2-7B among the models we test (as reported in QServe [30]). For the WOQ LUT comparison, we use FIGLUT, the SOTA ASIC LUT design evaluated at W4A16 precision.

Fig. 11 shows the normalized throughput and energy consumption of OASIS and baseline accelerators in single-batch decoding, with results normalized to FIGLUT. N.S. indicates

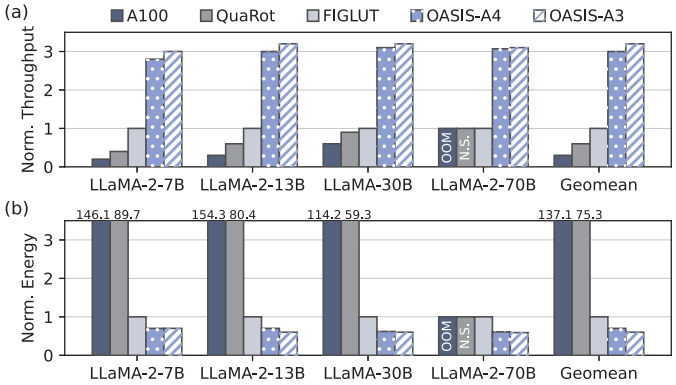


Fig. 11. Normalized throughput and energy consumption of OASIS and baseline accelerators in single-batch decoding.

that the accelerator does not support the corresponding model, while OOM indicates that the accelerator runs out of memory for the specified model. For throughput, on average, OASIS-A4 achieves 5.41×, 3.12×, 3.00× speedup and OASIS-A3 achieves 5.67×, 3.27×, 3.15× speedup over A100, QuaRot, and FIGLUT, respectively. For energy efficiency, on average, OASIS-A4 achieves 198.1×, 108.8×, 1.44×, and OASIS-A3 achieves 206.53×, 113.56×, 1.51× energy efficiency improvement over A100, QuaRot, and FIGLUT, respectively. The performance of GPU-based accelerators (A100 and QuaRot) is limited by low batch sizes during single-batch decoding, while FIGLUT's performance is constrained by limited parallelism due to small group sizes. In contrast, OASIS leverages an efficient WAQ LUT-GEMM design to substantially enhance computational parallelism, yielding superior throughput and energy efficiency.

### D. Ablation Studies

*1) Batched-Decoding:* OASIS is an ASIC accelerator tar-

TABLE IV
ZERO-SHOT ACCURACY RESULTS WITH 2048 SEQUENCE LENGTH.

| Model | Precision | Method | Zero-Shot Accuracy ↑ | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | | PIQA | ARC-E | ARC-C | BoolQ | HellaSwag | WinoGrande | Avg. |
| LLaMA-2-7B | **FP16** | – | 78.67 | 74.58 | 46.16 | 78.59 | 75.95 | 68.98 | 70.49 |
| | **W4A4** | QuaRot | 76.39 | 69.61 | 40.61 | 72.48 | 71.63 | 63.06 | 65.63 |
| | | Atom† | 75.14 | 52.99 | 38.40 | 74.59 | 69.37 | 62.75 | 62.21 |
| | | OASIS-S | 77.31 | 71.46 | 42.92 | 76.06 | 72.57 | 64.80 | 67.52 |
| | | **OASIS** | **77.97** | **73.06** | **43.60** | **76.83** | **74.32** | **65.51** | **68.55** |
| | **W4A3** | QuaRot | 53.16 | 27.99 | 25.26 | 41.10 | 28.75 | 49.49 | 37.63 |
| | | Atom† | 71.01 | 48.63 | 33.49 | 58.73 | 62.54 | 59.50 | 55.65 |
| | | OASIS-S | 75.14 | 63.93 | 37.37 | 63.89 | 67.58 | 63.93 | 61.97 |
| | | **OASIS** | **75.84** | **65.99** | **39.59** | **65.47** | **68.28** | **64.17** | **63.22** |
| LLaMA-3-8B | **FP16** | – | 80.63 | 77.62 | 57.71 | 81.28 | 79.61 | 73.70 | 75.09 |
| | **W4A4** | QuaRot | 68.28 | 60.48 | 37.46 | 66.57 | 61.73 | 63.06 | 59.60 |
| | | Atom† | 69.45 | 63.26 | 40.12 | 67.67 | 69.75 | 61.13 | 61.90 |
| | | OASIS-S | 77.62 | 73.95 | 50.34 | 78.67 | 75.88 | 70.56 | 71.17 |
| | | **OASIS** | **78.67** | **74.03** | **51.37** | **80.02** | **77.00** | **71.27** | **72.06** |
| | **W4A3** | QuaRot | 49.84 | 26.18 | 25.60 | 43.82 | 26.09 | 50.20 | 36.95 |
| | | Atom† | 72.86 | 51.06 | 40.52 | 61.19 | 67.78 | 60.87 | 59.05 |
| | | OASIS-S | 75.82 | 70.22 | 41.99 | 74.01 | 71.98 | 65.03 | 66.51 |
| | | **OASIS** | **77.09** | **71.89** | **45.65** | **75.64** | **73.80** | **66.22** | **68.38** |
| Mistral | **FP16** | – | 82.54 | 79.42 | 54.18 | 77.39 | 81.18 | 75.22 | 74.99 |
| | **W4A4** | QuaRot | 80.19 | 70.97 | 41.06 | 73.00 | 72.88 | 72.34 | 68.41 |
| | | Atom† | 80.71 | 68.63 | 52.39 | 74.55 | 77.52 | 72.03 | 70.97 |
| | | OASIS-S | 81.77 | 77.26 | 51.81 | 74.20 | 78.99 | 72.97 | 72.83 |
| | | **OASIS** | **82.10** | **77.82** | **53.24** | **75.77** | **80.15** | **73.40** | **73.80** |
| | **W4A3** | QuaRot | 53.16 | 27.99 | 25.26 | 41.10 | 25.68 | 48.78 | 36.99 |
| | | Atom† | 73.69 | 54.06 | 37.98 | 68.07 | 73.24 | 63.73 | 61.80 |
| | | OASIS-S | 77.35 | 74.01 | 43.68 | 70.85 | 76.02 | 66.93 | 68.14 |
| | | **OASIS** | **79.87** | **76.30** | **49.32** | **73.03** | **78.58** | **70.56** | **71.28** |

† Atom applies group quantization to weights and activations, with the group size of 128.

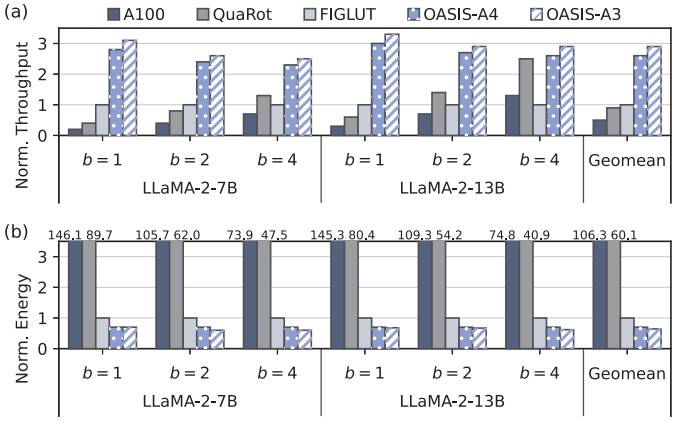


Fig. 12. Normalized throughput and energy consumption of OASIS and baseline accelerators during low-batch decoding.

geting edge LLM inference, where low-batch decoding is the predominant use case. In Fig. 12, we compare the normalized throughput and energy consumption of OASIS-A4/A3 over the baseline accelerators during low-batch decoding with batch sizes of 1, 2, and 4. The evaluation is conducted with the LLaMA-2-7B/13B models. OASIS-A4/A3 achieve average speedups of 3.41× and 3.73× over baseline accelerators, and average energy efficiency improvements of 26.43× and 28.20×, respectively. As the batch size increases, all accelerators exhibit higher throughput and lower energy consumption, primarily due to increased arithmetic intensity from weight reuse. GPU-based approaches show steady throughput gains as batch size increases, which is because of higher Tensor Core utilization on GPUs [39]. Nonetheless, OASIS still surpasses the baseline accelerators in both throughput and energy efficiency, especially with the smaller model of LLaMA-2-7B, which is more relevant for edge deployment.

*2) Prefill vs Decode:* We evaluate the performance of OASIS and FIGLUT under different prefill and decode length pairs using the LLaMA-2-7B/70B models, which is shown in Fig. 13. On average, OASIS-A4/A3 achieves 2.80× and 2.93× speedup over FIGLUT across different prefill/decode length pairs. Notably, OASIS's throughput and energy efficiency improvement over FIGLUT is more pronounced on the LLaMA-2-70B model than on the LLaMA-2-7B model, which is because larger models have a higher number of input channels, allowing OASIS to better leverage its compute efficiency advantage.

*3) Cycle Latencies for each Step in the Computation Pipeline:* Fig. 14 shows the pipeline execution schedule of performing a 1-4096-4096 GEMM with OASIS at W4A4 preci-

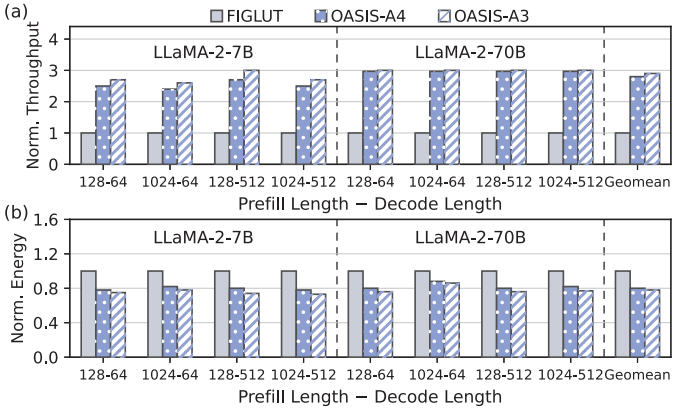


Fig. 13. Normalized throughput and energy consumption of OASIS and baseline accelerators for various prefill/decode length pairs.

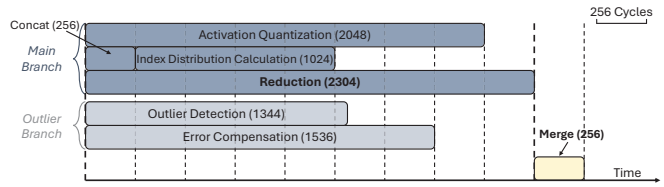


Fig. 14. Computation pipeline of performing an 1-4096-4096 GEMM with 1% outliers on OASIS at W4A4 precision. The numbers in parentheses indicate the number of cycles required for each step. The steps that bottleneck each pipeline stage are bolded.

sion with 1% outliers. The cycle latencies of each step are also shown in the figure with the numbers in parentheses. Based on the hardware configurations in Table II, in the 1% outlier case, the two branches exhibit comparable latencies, with the outlier branch completing approximately 33% faster. Consequently, the outlier branch finishes first and outputs results to the Output Buffer, which are subsequently merged with the main branch results upon completion. Conversely, in outlier-heavy scenarios, the main branch may finish first, with its results held in the Output Buffer awaiting the outlier branch completion.

*4) Outlier Sensitivity:* Fig. 15(a) presents the WikiText-2 PPL of LLaMA-2-7B and Mistral-7B on OASIS for outlier percentages ranging from 0.5% to 10%. For both models, increasing the outlier percentage generally improves PPL. To further examine the impact of increasing the outlier percentage on throughput, Fig. 15(b) and (c) show the throughput of LLaMA-2-7B and Mistral-7B normalized to that of OASIS-A4, respectively. We make two observations: (i) increasing the outlier percentage from 0.5% to 1% results in negligible throughput degradation for both models, as the end-to-end latency is dominated by the main branch; (ii) further increasing the outlier percentage from 1% to 10% leads to a significant increase in the execution time of the outlier branch, which becomes the new bottleneck of the end-to-end latency. This is because, as discussed in § IV-A, the hardware configurations in Table II are chosen such that the execution times of the main and outlier branches are comparable at 1% outlier percentage. Therefore, when the outlier percentage remains at or below 1%, the outlier branch does not constitute a bottleneck; however, once it exceeds this threshold, the computational

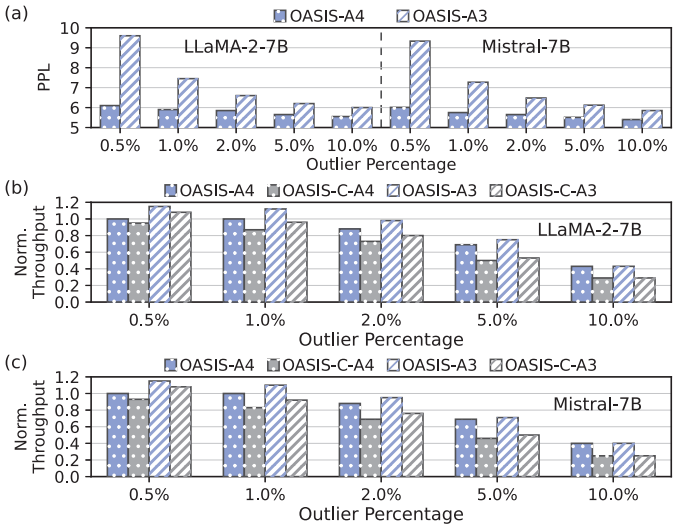


Fig. 15. (a) PPL, (b) LLaMA-2-7B's normalized throughput, and (c) Mistral-7B's normalized throughput of OASIS across different outlier percentages.

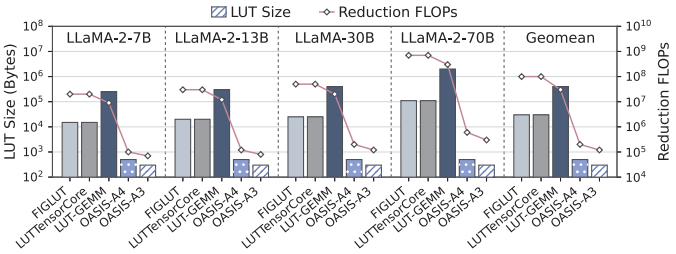


Fig. 16. LUT sizes and reduction FLOPs of OASIS and WOQ LUT-GEMM designs for the GEMM of the $q\_proj$ layer.

overhead of the outlier branch grows rapidly and dominates the overall latency.

To demonstrate the effectiveness of the look-ahead design, we quantify the latency of dynamic outlier detection by comparing OASIS's throughput to the conventional dynamic detection design (Fig. 4(a), denoted as OASIS-C), where outlier detection is placed on the GEMM critical path. On LLaMA-2-7B, when keeping 1% of outliers, OASIS-A4 and OASIS-A3 achieve 16% and 18% higher throughput than OASIS-C-A4 and OASIS-C-A3, respectively, demonstrating the importance of the look-ahead design in hiding the latency of dynamic outlier detection and achieving high throughput.

*5) Comparisons with LUT-Based GEMM Designs:* In Fig. 16, we compare the LUT sizes and FLOPs during reduction of OASIS with WOQ LUT-GEMM designs, including FIGLUT [42], LUT Tensor Core [37], and LUT-GEMM [43]. The evaluation is conducted on the $q\_proj$ layer's GEMM operation in different LLaMA models with W4A16 precision for WOQ LUT-GEMM designs. On average, OASIS-A4 reduces LUT sizes by 62.1×, and 994.2× compared to FIGLUT/LUT Tensor Core, and LUT-GEMM, respectively. OASIS-A4 also decreases FLOPs during reduction by 497.1×, and 248.6× compared to FIGLUT/LUT Tensor Core, and LUT-GEMM, respectively. The three LUT baseline methods all employ Inner Product LUTs with small group sizes to limit LUT size.

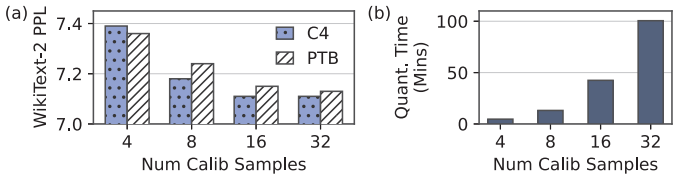


Fig. 17. Effects across calibration datasets and numbers of calibration samples on (a) PPL and (b) quantization time of OASIS-A4 on LLaMA-3-8B.

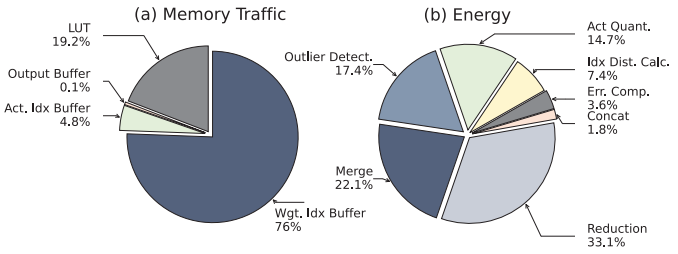


Fig. 18. Breakdown of (a) memory traffic and (b) energy consumption of OASIS-A4 for a 1-4096-4096 GEMM with 1% outliers.

This results in high FLOPs during reduction and consequently limits compute efficiency. Among these, LUT-GEMM trades off LUT size for lower FLOPs during reduction by utilizing a larger group size. As the model size increases from 7B to 70B, the number of input channels also increases from 4096 to 26728, leading to a significant rise in LUT sizes for all WOQ LUT-GEMM designs. In contrast, OASIS adopts Cartesian Product LUTs, which enable constant LUT sizes regardless of the number of input channels. As the model size increases, the increase of FLOPs in OASIS during reduction is also marginal compared to WOQ LUT-GEMM designs.

*6) Robustness of Offline-Learned Activation Centroids:* Fig. 17 investigates how calibration dataset selection and sample quantity affect the PPL and quantization time of OASIS-A4 on LLaMA-3-8B. As shown in Fig. 17(a), PPL remains consistent across different calibration datasets (C4 and PTB), with minimal variation. For instance, at 16 samples, PPL is 7.11 (C4) versus 7.15 (PTB). Generally, using C4 as the calibration dataset yielding slightly better PPL than PTB, which is because C4 is a larger and more comprehensive dataset than PTB, providing better coverage of the data distribution for centroid learning. Increasing calibration samples from 4 to 32 improves PPL (7.39 $\rightarrow$ 7.11 for C4), but convergence occurs around 16 samples, beyond which quantization time grows substantially (42.47 $\rightarrow$ 100.52 minutes) without significant PPL gains. Consequently, we employ 16 C4 samples for activation centroid learning in OASIS to achieve an optimal balance between accuracy and efficiency.

*7) Memory access / energy breakdown:* In Fig. 18, we present the breakdown of on-chip memory traffic and energy consumption for a 1-4096-4096 GEMM with 1% outliers with OASIS-A4. Memory traffic is measured as the total number of bytes transferred, including both reads and writes. The Weight Index Buffer dominates memory traffic at 76.0%, while LUT reads and writes contribute 19.2%, demonstrating that LUT access does not induce significant memory overhead. Energy consumption is primarily attributed to reduction (33.1%) and merging results from the main and outlier branches (22.1%).

## VI. Related Works

### A. LLM WAQ Methods

In WAQ settings, both weights and activations are quantized to low precision, which can significantly reduce memory usage and computational costs during LLM inference [3], [14], [33], [62]. For example, SmoothQuant [58] applies scaling on both weights and activations to migrate the quantization difficulties of activations to weights, which are easier to quantize due to their smaller magnitude and quantity of outliers. QuaRot [3] applies Hadamard rotation matrices on both weights and activations to spread the quantization noise across all dimensions. Atom [62] applies fine-grained quantization granularity to limit the impact of quantization noise caused by outliers within smaller groups, and preserve some outliers with higher precision. However, they still lead to noticeable PPL degradation compared to FP16 models in low-precision configurations, and induce additional runtime overhead during GEMM operations. In contrast, OASIS does not incorporate outlier suppression operations, and handle the outliers without additional runtime overhead.

### B. Reduction Tree-Based Architectures

Reduction trees perform O(N) operations with O(logN) latency by exploiting parallelism across tree levels. They are widely used for summation, e.g., in MAERI [23] and Flexagon [38], and can also support outlier selection via tournament trees [49], which identify maxima or minima through hierarchical pairwise comparisons. Inspired by tournament trees, we develop *Orizuru*, an outlier detection engine tailored for efficiently identifying both maximum and minimum activation outliers during LLM inference. *Orizuru* features shared leaf nodes between the maximum and minimum trees, which allows for efficient comparison of both maximum and minimum values with reduced hardware costs.

## VII. Conclusion

OASIS introduces an approach to executing NU-WAQ inference by eliminating dequantization and maximizing compute efficiency. By leveraging offline-computed Cartesian-product LUTs, OASIS significantly reduces LUT sizes and enables large-granularity GEMMs that exploit massive parallelism. Its outlier-aware quantization and lightweight *Orizuru* top-$k$ engine further preserve accuracy and efficiency without adding runtime latency and with only marginal energy overhead. Together, these innovations bridge the algorithm-hardware gap for NU-WAQ, maintaining accuracy with substantial throughput and energy efficiency gains.

## Acknowledgement

This work was partially supported by NSF under Grant Nos. 2332744, 2112562, and 2148253, and by AFOSR under Grant No. FA9550-24-1-0322. The authors would like to thank Duke CEI Lab for their support. We also acknowledge helpful discussions with Yiran Chen, Zhixu Du and Changchun Zhou.

## REFERENCES


[1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[2] E. Alvarez, O. Almog, E. Chung, S. Layton, D. Stosic, R. Krashinsky, and K. Aubrey, "Introducing nvfp4 for efficient and accurate low-precision inference," https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/, Jun 2025.

[3] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, "Quarot: Outlier-free 4-bit inference in rotated llms," *arXiv preprint arXiv:2404.00456*, 2024.

[4] Y. Bisk, R. Zellers, J. Gao, Y. Choi *et al.*, "Piqa: Reasoning about physical commonsense in natural language," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 7432–7439.

[5] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[6] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "Boolq: Exploring the surprising difficulty of natural yes/no questions," in *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)*, 2019, pp. 2924–2936.

[7] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? try arc, the ai2 reasoning challenge," *arXiv preprint arXiv:1803.05457*, 2018.

[8] N. Corporation, "Nvidia rtx blackwell gpu architecture: Built for neural rendering," NVIDIA Corporation, Tech. Rep. V1.1, 2025, white paper. [Online]. Available: https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf

[9] J. Dodge, M. Sap, A. Marasović, W. Agnew, G. Ilharco, D. Groeneveld, M. Mitchell, and M. Gardner, "Documenting large webtext corpora: A case study on the colossal clean crawled corpus," *arXiv preprint arXiv:2104.08758*, 2021.

[10] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[11] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[12] Y. Fu, Y. Zhang, Z. Yu, S. Li, Z. Ye, C. Li, C. Wan, and Y. C. Lin, "Gpt4aigchip: Towards next-generation ai accelerator design automation via large language models," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.

[13] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, "The language model evaluation harness," 07 2024. [Online]. Available: https://zenodo.org/records/12608602

[14] Z. Gao, S. K. Vadlamani, K. Sulimany, D. Englund, and T. Chen, "Disaggregated machine learning via in-physics computing at radio frequency," *Science Advances*, vol. 12, no. 2, p. eadz0817, 2026.

[15] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[16] C. Guo, F. Cheng, Z. Du, J. Kiessling, J. Ku, S. Li, Z. Li, M. Ma, T. Molom-Ochir, B. Morris *et al.*, "A survey: Collaborative hardware and software design in the era of large language models," *IEEE Circuits and Systems Magazine*, vol. 25, no. 1, pp. 35–57, 2025.

[17] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[18] Z. He, H. Wu, X. Zhang, X. Yao, S. Zheng, H. Zheng, and B. Yu, "Chateda: A large language model powered autonomous agent for eda," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE, 2023, pp. 1–6.

[19] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "Kvquant: Towards 10 million context length llm inference with kv cache quantization," *arXiv preprint arXiv:2401.18079*, 2024.

[20] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.

[21] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, "Squeezellm: Dense-and-sparse quantization," *arXiv preprint arXiv:2306.07629*, 2023.

[22] T. Kumar, Z. Ankner, B. F. Spector, B. Bordelon, N. Muennighoff, M. Paul, C. Pehlevan, C. Ré, and A. Raghunathan, "Scaling laws for precision," *arXiv preprint arXiv:2411.04330*, 2024.

[23] H. Kwon, A. Samajdar, and T. Krishna, "Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects," *ACM Sigplan Notices*, vol. 53, no. 2, pp. 461–475, 2018.

[24] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.

[25] J. Li, J. Xu, S. Li, S. Huang, J. Liu, Y. Lian, and G. Dai, "Fast and efficient 2-bit llm inference on gpu: 2/4/16-bit in a weight matrix with asynchronous dequantization," *arXiv preprint arXiv:2311.16442*, 2023.

[26] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "Dramsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.

[27] S. Li, K. Chen, J. H. Ahn, J. B. Brockman, and N. P. Jouppi, "Cacti-p: Architecture-level modeling for sram-based structures with advanced leakage reduction techniques," in *2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2011, pp. 694–701.

[28] H. Lin, H. Xu, Y. Wu, J. Cui, Y. Zhang, L. Mou, L. Song, Z. Sun, and Y. Wei, "Duquant: Distributing outliers via dual transformation makes stronger quantized llms," *Advances in Neural Information Processing Systems*, vol. 37, pp. 87 766–87 800, 2025.

[29] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.

[30] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "Qserve: W4a8kv4 quantization and system co-design for efficient llm serving," *arXiv preprint arXiv:2405.04532*, 2024.

[31] S.-y. Liu, Z. Liu, X. Huang, P. Dong, and K.-T. Cheng, "Llm-fp4: 4-bit floating-point quantized transformers," *arXiv preprint arXiv:2310.16836*, 2023.

[32] W. Liu, H. Meng, Y. Luo, P. Zhang, and X. Ma, "Micromix: Efficient mixed-precision quantization with microscaling formats for large language models," *arXiv preprint arXiv:2508.02343*, 2025.

[33] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, and T. Blankevoort, "Spinquant–llm quantization with learned rotations," *arXiv preprint arXiv:2405.16406*, 2024.

[34] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proceedings of 5-th Berkeley Symposium on Mathematical Statistics and Probability/University of California Press*, 1967.

[35] M. Marcus, G. Kim, M. A. Marcinkiewicz, R. MacIntyre, A. Bies, M. Ferguson, K. Katz, and B. Schasberger, "The penn treebank: Annotating predicate argument structure," in *Human Language Technology: Proceedings of a Workshop held at Plainsboro, New Jersey, March 8-11, 1994*, 1994.

[36] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," *arXiv preprint arXiv:1609.07843*, 2016.

[37] Z. Mo, L. Wang, J. Wei, Z. Zeng, S. Cao, L. Ma, N. Jing, T. Cao, J. Xue, F. Yang *et al.*, "Lut tensor core: A software-hardware co-design for lut-based low-bit llm inference," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 514–528.

[38] F. Muñoz-Martínez, R. Garg, M. Pellauer, J. L. Abellán, M. E. Acacio, and T. Krishna, "Flexagon: A multi-dataflow sparse-sparse matrix multiplication accelerator for efficient dnn processing," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023, pp. 252–265.

[39] NVIDIA, "Tensor core performance: The ultimate guide," NVIDIA, Tech. Rep., 2019.

[40] ——, "Nvidia a100 tensor core gpu architecture," NVIDIA, Tech. Rep., 2020.

[41] NVIDIA Corporation, "Nvidia turing gpu architecture whitepaper," NVIDIA Corporation, Tech. Rep. 87 pages, 2018. [Online]. Available: https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf

[42] G. Park, H. Kwon, J. Kim, J. Bae, B. Park, D. Lee, and Y. Lee, "Figlut: An energy-efficient accelerator design for fp-int gemm using look-up tables," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1098–1111.

[43] G. Park, B. Park, M. Kim, S. Lee, J. Kim, B. Kwon, S. J. Kwon, B. Kim, Y. Lee, and D. Lee, "Lut-gemm: Quantized matrix multiplication based on luts for efficient inference in large-scale generative language models," *arXiv preprint arXiv:2206.09557*, 2022.

[44] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[45] J. Pennington and P. Worah, "The spectrum of the fisher information matrix of a single-hidden-layer neural network," *Advances in neural information processing systems*, vol. 31, 2018.

[46] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[47] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, "Winogrande: An adversarial winograd schema challenge at scale," *Communications of the ACM*, vol. 64, no. 9, pp. 99–106, 2021.

[48] H. Sharma, J. Park, D. Mahajan, E. Amaro, J. K. Kim, C. Shao, A. Mishra, and H. Esmaeilzadeh, "From high-level deep neural models to fpgas," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–12.

[49] A. A. Stepanov and A. Kershenbaum, "Using tournament trees to sort," Center for Advanced Technology in Telecommunications, Polytechnic University of New York, Tech. Rep. 86-13, 1986.

[50] Y. Sun, R. Liu, H. Bai, H. Bao, K. Zhao, Y. Li, J. Hu, X. Yu, L. Hou, C. Yuan *et al.*, "Flatquant: Flatness matters for llm quantization," *arXiv preprint arXiv:2410.09426*, 2024.

[51] T. Tao, J. Li, B. Tan, H. Wang, W. Marshall, B. M. Kanakiya, J. Hestness, N. Vassilieva, Z. Shen, E. P. Xing *et al.*, "Crystal: Illuminating llm abilities on language and code," *arXiv preprint arXiv:2411.04156*, 2024.

[52] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[53] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[54] A. Tseng, T. Yu, and Y. Park, "Training llms with mxfp4," *arXiv preprint arXiv:2502.20586*, 2025.

[55] H. Wang, Z. Zhang, and S. Han, "Spatten: Efficient sparse attention architecture with cascade token and head pruning," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 97–110.

[56] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.

[57] X. Wu, E. Hanson, N. Wang, Q. Zheng, X. Yang, H. Yang, S. Li, F. Cheng, P. P. Pande, J. R. Doppa, K. Chakrabarty, and H. Li, "Block-wise mixed-precision quantization: Enabling high efficiency for practical reram-based dnn accelerators," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 12, pp. 4558–4571, 2024.

[58] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.

[59] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "Hellaswag: Can a machine really finish your sentence?" *arXiv preprint arXiv:1905.07830*, 2019.

[60] D. Zhang, J. Yang, D. Ye, and G. Hua, "Lq-nets: Learned quantization for highly accurate and compact deep neural networks," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 365–382.

[61] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.

[62] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, "Atom: Low-bit quantization for efficient and accurate llm serving," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 196–209, 2024.